IV_SEM\Lab_1\BisectionMethod.py

```
'''
        By       :- Darshan Shrestha
        Roll no.:- KCE081BCT010
'''
import matplotlib.pyplot as plt
import numpy as np

TOLERABLE_ERROR = 10**-5

def func(x):
    return x**2 - 5 * x + 5


def plotFunc():
    x_values = np.linspace(0, 5, 200)
    y_values = [func(x) for x in x_values]

    plt.figure(figsize=(8, 4))
    plt.plot(x_values, y_values, label="f(x) = x^2 - 5x + 5")
    plt.axhline(0, color="black", linewidth=0.8)
    plt.xlabel("x")
    plt.ylabel("f(x)")
    plt.title("Plot of f(x)")
    plt.grid(True)
    plt.legend()
    plt.show()

def print_iteration_table(rows) -> None:
    columns = ["Iteration", "a", "b", "m", "f(a)", "f(m)", "Error"]
    widths = {
        "Iteration": len("Iteration"),
        "a": len("a"),
        "b": len("b"),
        "m": len("m"),
        "f(a)": len("f(a)"),
        "f(m)": len("f(m)"),
        "Error": len("Error"),
    }

    for row in rows:
        widths["Iteration"] = max(widths["Iteration"], len(f"{row['Iteration']}"))
        for key in ("a", "b", "m", "f(a)", "f(m)", "Error"):
            widths[key] = max(widths[key], len(f"{row[key]:.6f}"))

    header = " | ".join(f"{column:>{widths[column]}}" for column in columns)
    separator = "-+-".join("-" * widths[column] for column in columns)

    print("\nBy      :- Darshan Shrestha")
    print("Roll no. :- KCE081BCT010")
    print("\nIteration Table:\n")
    print(header)
```

```
    print(separator)
    for row in rows:
        print(
            f"{row['Iteration']:>{widths['Iteration']}} | "
            f"{row['a']:>{widths['a']}.6f} | "
            f"{row['b']:>{widths['b']}.6f} | "
            f"{row['m']:>{widths['m']}.6f} | "
            f"{row['f(a)']:>{widths['f(a)']}.6f} | "
            f"{row['f(m)']:>{widths['f(m)']}.6f} | "
            f"{row['Error']:>{widths['Error']}.6f}"
        )

def findRoot(a, b) -> list:
    iterations = 0
    rows = []
    while True:
        m = (a + b) / 2
        func_a = func(a)
        func_m = func(m)
        error = abs(b - a) / 2

        rows.append({
            "Iteration": iterations + 1,
            "a": a,
            "b": b,
            "m": m,
            "f(a)": func_a,
            "f(m)": func_m,
            "Error": error,
        })

        if func_m * func_a < 0:
            b = m
        else:
            a = m

        iterations += 1
        if abs(func_m) < TOLERABLE_ERROR:
            return [m, iterations, rows]

def main() -> None:
    check = True
    plotFunc();
    while check:
        print("f(x) = x**2 - 5 * x + 5\n")
        initGuess_1, initGuess_2  = map(float, input("Enter initial guesses"
        "(2 only; separate with spaces).=> ").split())

        if (func(initGuess_1) * func(initGuess_2) >= 0):
            print("Error; the product of the functional value of inputs must be strictly less than 0.\n")
        else:
            check = False

    a = initGuess_1
```

```
    b = initGuess_2

    root, iterations, rows = findRoot(a,b)
    print_iteration_table(rows)

    root_line = f"Root: {root:.5f}"
    iter_line = f"Ran {iterations} iterations"
    content_width = max(len(root_line), len(iter_line))
    border = "+" + "-" * (content_width + 2) + "+"
    print()
    print(border)
    print(f"| {root_line.ljust(content_width)} |")
    print(f"| {iter_line.ljust(content_width)} |")
    print(border)


if __name__ == "__main__":
    main()

```